Calibration et Simulation

TP1 : Calibration de volatilité implicite

STEPHAN Corentin

Table des matières

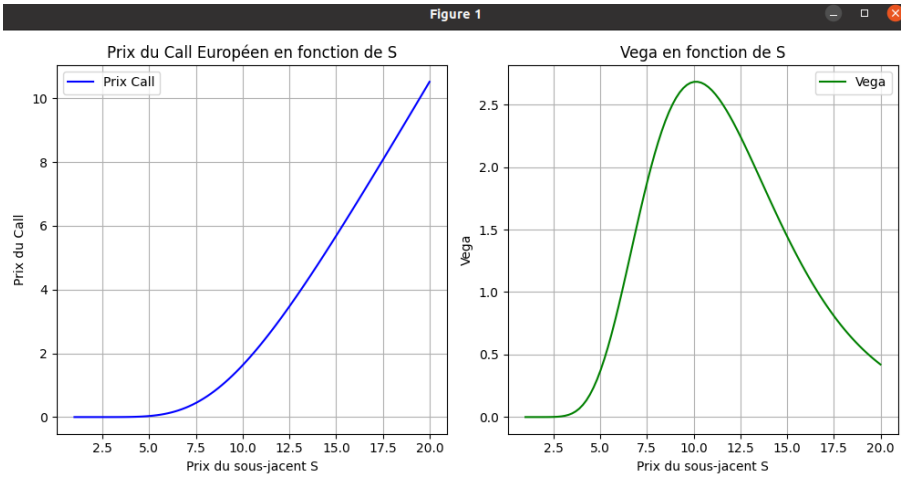
I - LIFFE **5**
Question 1 5
II - Base de données “Sp-index” **7**
Question 2 7
Call en 3D 7
Put en 3D 9
Superposition 10
Question 3 11
Call en 2D 11
Put en 2D 12
Question 4 13
III - Base de données “Google Origin” **16**
Question 5 16
IV - Base de données “Spx-quotedata” **18**
Question 6-7-8 18
Question 9 23
V- Base de données "new-Data.csv" **25**
Question 10-11 25

Fonctions supports :

```python
#On met ici toutes les fonctions utiles à la suite du TP
def BlackScholes_Call(S, K, T, t, r, sigma):
    d1 = (np.log(S / K) + (r + 0.5 * sigma**2) * (T-t)) / (sigma * np.sqrt(T-t))
    d2 = d1 - sigma * np.sqrt(T - t)
    call_price = S * norm.cdf(d1) - K * np.exp(-r * (T-t)) * norm.cdf(d2)
    return call_price


def BlackScholes_Put(S, K, T, t, r, sigma):
    put_price = BlackScholes_Call(S, K, T, t, r, sigma)- S + K * np.exp(-r * (T-t))
    return put_price

def Vega(S, K, T, t, r, sigma):
    d1 = (np.log(S / K) + (r + 0.5 * sigma**2) * (T-t)) / (sigma * np.sqrt(T-t))
    vega = S * norm.pdf(d1) * np.sqrt(T-t)
    return vega

#Algorithme de Newton
def Newton(S,K,r,sigma,T,t,M,g):
    while abs((F(S,K,r,sigma,T,t,M))) > g:
        sigma = sigma - F(S,K,r,sigma,T,t,M)/Vega(S, K, T, t, r, sigma)
    return sigma

def F(S,K,r,sigma,T,t,M):
    return BlackScholes_Call(S, K, T, t, r, sigma) - M

def F_Put(S, K, r, sigma, T, t, M):
    return BlackScholes_Put(S, K, T, t, r, sigma) - M
```

Quelques explications :

- fonction V_BS : fonction utilisée pour la représentation des Call



On a aussi réalisé en question préliminaire le calcul du prix d'un Call via la formule de Black&Scholes ainsi que le calcul du vega utilisé par la fonction ci-dessus, et pour le plot nous avons utilisé:

```python
# Liste de prix du sous-jacent
S_values = np.linspace(1, 20, 100)
call_prices = [BlackScholes_Call(S, K, T, t, r, sigma) for S in S_values]
vega_values = [Vega(S, K, T, t, r, sigma) for S in S_values]


# Liste de prix du sous-jacent
S_values = np.linspace(1, 20, 100)
call_prices = [BlackScholes_Call(S, K, T, t, r, sigma) for S in S_values]
vega_values = [Vega(S, K, T, t, r, sigma) for S in S_values]

# Plot Call
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.plot(S_values, call_prices, label="Prix Call", color='blue')
plt.title("Prix du Call Européen en fonction de S")
plt.xlabel("Prix du sous-jacent S")
plt.ylabel("Prix du Call")
plt.grid(True)
plt.legend()

# Plot Vega
plt.subplot(1, 2, 2)
plt.plot(S_values, vega_values, label="Vega", color='green')
plt.title("Vega en fonction de S")
plt.xlabel("Prix du sous-jacent S")
plt.ylabel("Vega")
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()
```

```python
# Fonction de vérification pour le Call
def verification(Call, S, K, T, t, r):
    if max(S - K * np.exp(-r * (T-t)), 0) < Call < S:
        return 1
    return 0

# Fonction de vérification du Put
def verification2(Put, S, K, T, t, r):
    if max(S - K * np.exp(-r * (T-t)),0) - S + K*np.exp(-r*(T-t)) < Put < K * np.exp(-r*(T-t)):
        return 1
    return 0
```

- fonction verification : respecter contraintes dans le cas d'un Call
- fonction verification2 : respecter contraintes dans le cas d'un Put. Pour trouver un encadrement d'un Put, nous avons utilisé à la fois les contraintes d'un Call et la relation de parité Call-Put :

$$Call(S_0, T, K) - Put(S_0, T, K) = S_0 . e^{-q.T} - K.e^{-r.T}$$

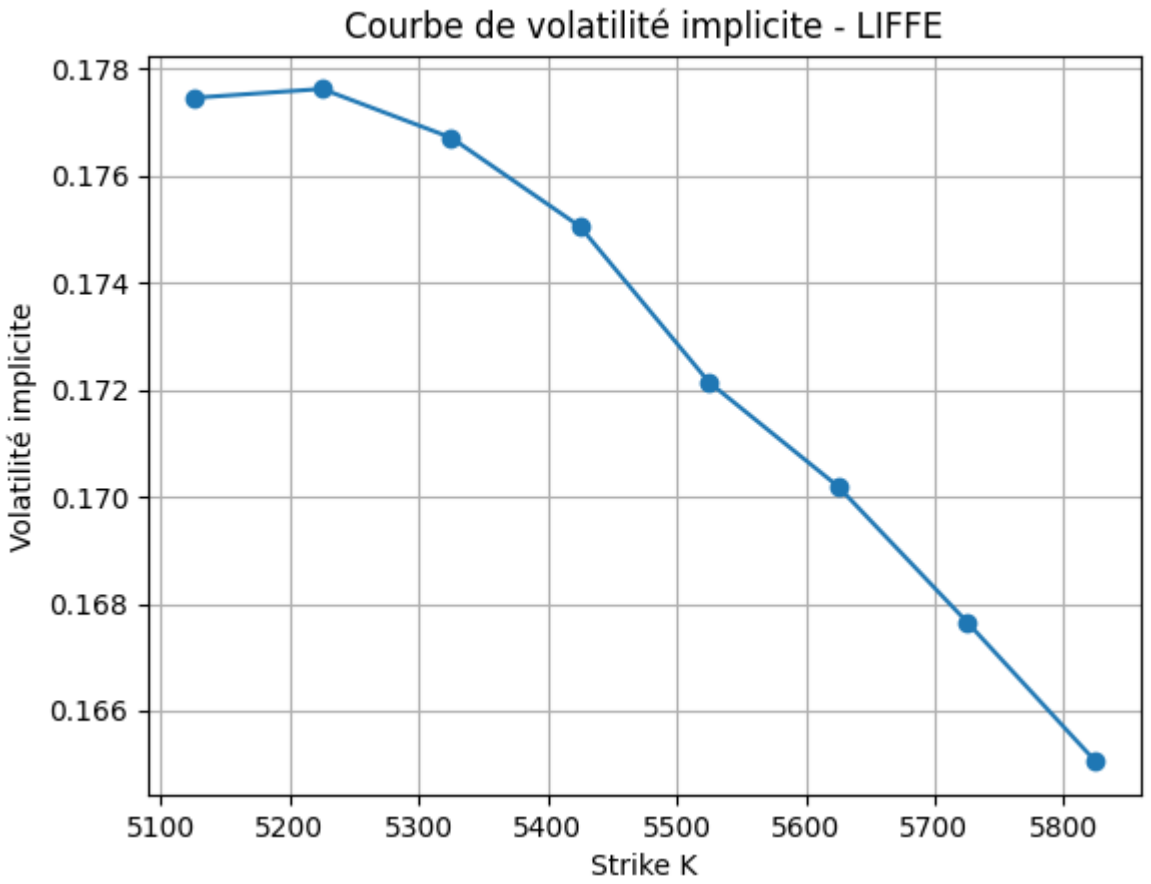
I - LIFFE

Question 1

Code :

```
#Travail 1 LIFFE
def Part1():
    t = 4 / 12
    T = 25 / 36
    S0 = 5430.3
    r = 0.05
    M = [475, 405, 340, 280.5, 226, 179.5, 139, 105]
    k = [5125, 5225, 5325, 5425, 5525, 5625, 5725, 5825]
    sigma_imp = [0] * len(k)
    g = 0.0217

    # Volatilité implicite initiale
    sigma = np.sqrt(2 * abs((np.log(S0 / k[0]) + r * (T - t)) / (T - t)))

    for i in range(8):
        sigma = Newton(S0, k[i], r, sigma, T, t, M[i], g)
        sigma_imp[i] = sigma

    plt.plot(k, sigma_imp, marker='o')
    plt.xlabel("Strike K")
    plt.ylabel("Volatilité implicite")
    plt.title("Courbe de volatilité implicite - LIFFE")
    plt.grid(True)
    plt.show()
```

Graphique affiché :



Interprétation :

Nous constatons que plus le prix du strike K est élevé, plus la volatilité implicite diminue. En termes financiers, ce phénomène est observé sur des marchés dans lesquels les options d'achat hors de la monnaie avec un strike élevé sont "moins" risquées que les options dans la monnaie ou à la monnaie. Le smile permet également d'anticiper les fluctuations du marché. Lorsque la volatilité implicite est plus élevée et donc des strikes plus bas, nous pourrions avoir un risque plus important sur une possible baisse du prix de l'actif sous-jacent. Cependant, ce n'est pas fixé et il faut prendre en considération l'ensemble des évènements du marché.

Pour rappel :

- dans la monnaie : option pour laquelle son exercice pourrait être avantageux si exercée directement
  → prix actif sous-jacent > prix strike K (Call)
  → prix actif sous-jacent < prix strike K (Put)
- hors de la monnaie : option pour laquelle son exercice ne pourrait pas être avantageux si exercée directement
  → prix actif sous-jacent < prix strike K (Call)
  → prix actif sous-jacent > prix strike K (Put)
- à la monnaie : option pour laquelle le prix de l'actif sous-jacent est proche du prix d'exercice

II - Base de données “Sp-index”

Question 2

Call en 3D

Code :

```
# Graphique du Call 3D

def Call_3D():
    eps = 0.0001
    t = 0
    q = 0.0217
    S0 = 1260.36

    chemin = "/home/cytech/Desktop/Documents/ING3/Calibration/TP1/sp-index.txt"
    fichier = np.loadtxt(chemin,skiprows=1)
    N = len(fichier) # Calcul du nb de lignes pour initialiser tous les vecteurs de données

    # Initialisation des données
    S_0 = np.zeros(N)
    vol_imp = np.zeros(N)
    K = np.zeros(N)
    r = np.zeros(N)
    T = np.zeros(N)
    Cb = np.zeros(N)
    Ca = np.zeros(N)
    Call = np.zeros(N)

    # Chargement des données
    T = fichier[:,0]
    K = fichier[:,1]
    Cb = fichier[:,2]
    Ca = fichier[:,3]
    r = fichier[:,6]/100

    #Calcul du cours
    Call = (Cb + Ca)/2

    for i in range(N):
        #Actualisation du prix
        S_0[i] = S0*np.exp(-q * T[i])
        if verification(Call[i],S_0[i],K[i],T[i], t, r[i]):
            sigma = []
            #Initialisation du sigma
            sigma.append(np.sqrt(2*abs((np.log(S_0[i]/K[i])+r[i]*T[i])/T[i])))
            #Algorithme de Newton
            while abs(F(S_0[i],K[i],r[i],sigma[-1],T[i],t,Call[i])) > eps:
                sigma.append(sigma[-1] - F(S_0[i],K[i],r[i],sigma[-1],T[i],t,Call[i])/Vega(S_0[i],K[i], T[i], t, r[i],sigma[-1]))
                vol_imp[i] = sigma[-1]
        else :
            vol_imp[i] = 0

    # Suppression de tout les strikes et sigma ou la volatilité implicite est nullle
    volat_nules=np.where(vol_imp==0)
    K = np.delete(K,volat_nules)
    T = np.delete(T,volat_nules)
    vol_imp = np.delete(vol_imp,volat_nules)

    fig = plt.figure()
    ax = fig.add_subplot(111, projection='3d')
    ax.scatter(K, T, vol_imp, c='b', marker='o')

    ax.set_title("Partie 3 Call 3D")
    ax.set_xlabel("K")
    ax.set_ylabel("T")
    ax.set_zlabel("Vol imp")

    plt.show()
```

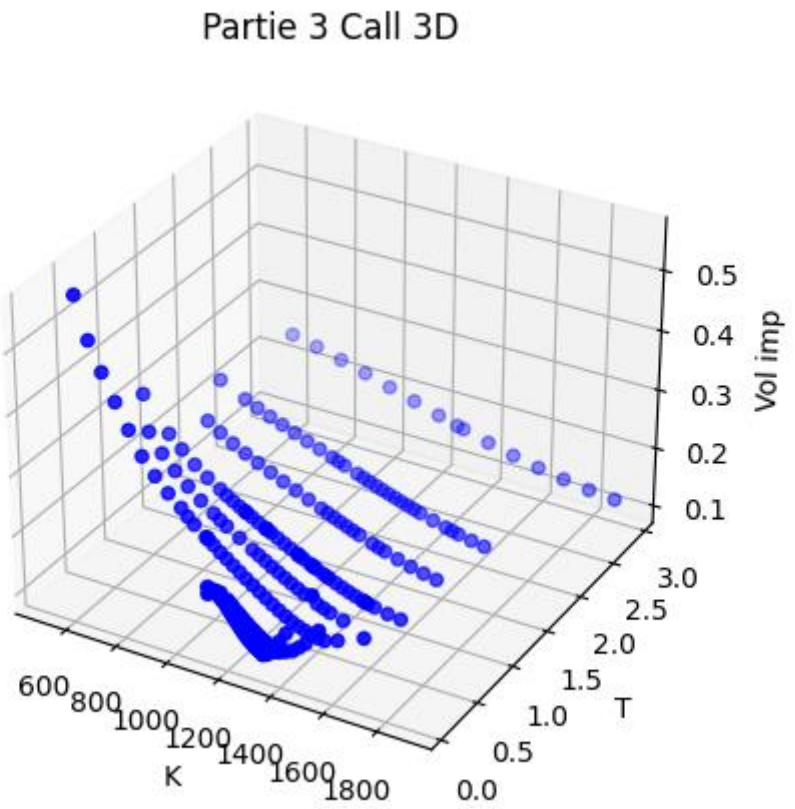
Explications :

Après avoir extrait les données, je vérifie les contraintes liées au Call pour chaque ligne du fichier spindex.txt. Lorsque la condition est vérifiée, nous utilisons la formule liée la volatilité, à savoir :

$$\sigma_0 = \sqrt{2|\frac{ln(S_0/K) + rT}{T}|}$$

Ensuite nous utilisons l'algorithme de Newton pour chaque ligne. Il faut par la suite supprimer les volatilités nulles. Il est nécessaire de vérifier que les valeurs de la maturité, du prix d'exercice strike et de la volatilité implicite soient de la même dimension.

Graphique affiché :



Interprétation :

Lorsque le prix d'exercice est proche de la valeur de S0, la volatilité implicite est plus faible, cela montre une incertitude moins importante sur les prochains mouvements du prix du sous-jacent.
En revanche, lorsque le prix d'exercice est éloigné de la valeur de S0, la volatilité implicite tend à être bien plus élevée, indiquant un risque plus important pour les variations du sous-jacent.

Put en 3D

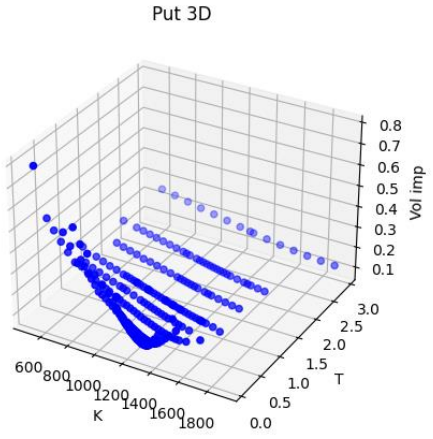
Code :

```python
# graphique du Put 3D

def Put_3D():
    eps = 0.0001
    t = 0
    q = 0.0217
    S0 = 1260.36

    chemin = "/home/cytech/Desktop/Documents/ING3/Calibration/TP1/sp-index.txt"
    fichier = np.loadtxt(chemin,skiprows=1)
    N = len(fichier) # Calcul du nb de lignes pour initialiser tous les vecteurs de données

    # Initialisation des données
    S_0 = np.zeros(N)
    vol_imp = np.zeros(N)
    K = np.zeros(N)
    r = np.zeros(N)
    T = np.zeros(N)
    Pb = np.zeros(N)
    Pa = np.zeros(N)
    Put = np.zeros(N)

    # Chargement des données
    T = fichier[:,0]
    K = fichier[:,1]
    Pb = fichier[:,4]
    Pa = fichier[:,5]
    r = fichier[:,6]/100

    # Calcul du cours
    Put = (Pb + Pa)/2

    for i in range(N):
        #Actualisation du prix
        S_0[i] = S0*np.exp(-q * T[i])
        if verification2(Put[i],S_0[i],K[i],T[i], t, r[i]):
            sigma = []
            #Initialisation du sigma
            sigma.append(np.sqrt(2*abs((np.log(S_0[i]/K[i])+r[i]*T[i])/T[i])))
            #Algorithme de Newton
            while abs(F_Put(S_0[i],K[i],r[i],sigma[-1],T[i],t,Put[i])) > eps:
                sigma.append(sigma[-1] - F_Put(S_0[i], K[i], r[i], sigma[-1], T[i], t, Put[i]) / Vega(S_0[i], K[i], T[i], t, r[i], sigma[-1]))
                vol_imp[i] = sigma[-1]
        else :
            vol_imp[i] = 0

    # Suppression de tout les strikes et sigma ou la volatilité implicite est nullle
    volat_nules=np.where(vol_imp==0)
    K = np.delete(K,volat_nules)
    T = np.delete(T,volat_nules)
    vol_imp = np.delete(vol_imp,volat_nules)

    fig = plt.figure()
    ax = fig.add_subplot(111, projection='3d')
    ax.scatter(K, T, vol_imp, c='b', marker='o')

    ax.set_title("Put 3D")
    ax.set_xlabel("K")
    ax.set_ylabel("T")
    ax.set_zlabel("Vol imp")

    plt.show()
```

Explications :

Fonctionnement analogue au Call mais avec les contraintes liées au Put. Pour calculer les valeurs des put de chaque ligne nous faisons la somme du Put *bid* et du Put *ask* divisé par 2. Ici, nous utilisons par conséquent les fonctions “verification2” pour les contraintes du Put et “F-put” dans l’algorithme de Newton. Nous affichons la volatilité implicite en fonction des maturités et des prix d’exercices strike.

Graphique affiché :



Interprétation :

Même phénomène que pour le Call :

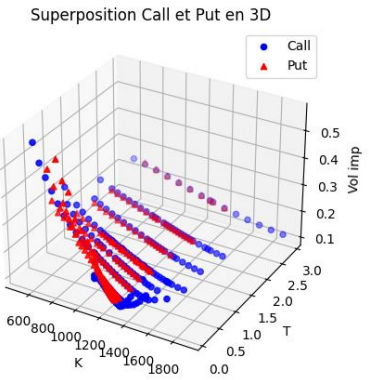
- Strike aux alentours du sous-jacent → faible volatilité implicite
- Strike éloigné du sous-jacent → forte volatilité implicite

Plus la maturité est éloignée, plus le phénomène se stabilise indiquant une intégration des risques par le marché

Superposition

Le code est une combinaison des deux précédents. Tel que : chargements des données, Calcul du Call puis Calcul du Put avec l'algorithme de Newton, ensuite on enlève les strike et les sigma là où la volatilité implicite est nulle

Graphique affiché :



Interprétation :

Nous pouvons constater que les smiles des Call et des Put se superposent très bien.

Question 3

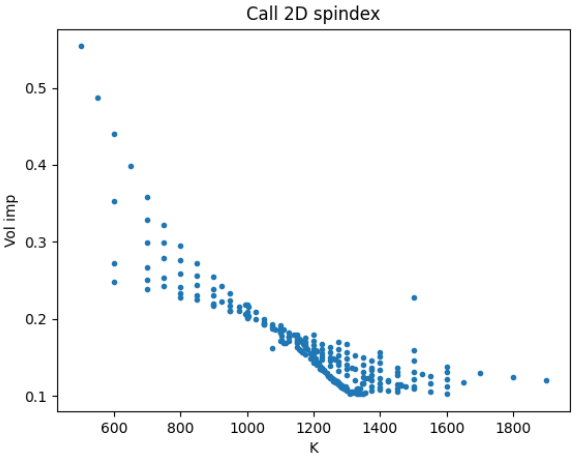
Call en 2D

Code :

```python
#Graphique du Call 2D
def Call_2D():
    eps = 0.0001  # seuil pour la vérification
    t = 0
    q = 0.0217
    S0 = 1260.36

    chemin = "/home/cytech/Desktop/Documents/ING3/Calibration/TP1/sp-index.txt"
    fichier = np.loadtxt(chemin,skiprows=1)
    N = len(fichier) # Calcul du nb de lignes pour initialiser tous les vecteurs de données

    #Initialisation des données
    S_0 = np.zeros(N)
    vol_imp = np.zeros(N)
    K = np.zeros(N)
    r = np.zeros(N)
    T = np.zeros(N)
    Cb = np.zeros(N)
    Ca = np.zeros(N)
    Call = np.zeros(N)

    #Calcul des données
    T = fichier[:,0]
    K = fichier[:,1]
    Cb = fichier[:,2]
    Ca = fichier[:,3]
    r = fichier[:,6]/100

    Call = (Cb + Ca)/2

    for i in range(N):
        S_0[i] = S0*np.exp(-q * (T[i] - t))
        if verification(Call[i],S_0[i],K[i],T[i], t, r[i]):
            sigma = []
            #Initialisation du sigma
            sigma.append(np.sqrt(2*abs((np.log(S_0[i]/K[i])+r[i]*(T[i] - t)/(T[i] - t)))))
            #Algorithme de Newton
            while abs(F(S_0[i],K[i],r[i],sigma[-1],T[i],t,Call[i])) > eps:
                sigma.append(sigma[-1] - F(S_0[i],K[i],r[i],sigma[-1],T[i],t,Call[i])/Vega(S_0[i],K[i], T[i], t, r[i],sigma[-1]))
                vol_imp[i] = sigma[-1]
        else :
            vol_imp[i] = 0

    # Suppression de tout les strikes et sigma ou la volatilité implicite est nullle
    volat_nules=np.where(vol_imp==0)
    K = np.delete(K,volat_nules)
    vol_imp = np.delete(vol_imp,volat_nules)

    plt.plot(K,vol_imp,".")
    plt.title("Call 2D spindex")
    plt.ylabel("Vol imp")
    plt.xlabel("K")
    plt.show()
```

Explications : Raisonnement analogue au Call en 3D.

Graphique affiché



Interprétation :

Plus le prix d'exercice est éloigné du sous-jacent, plus la volatilité implicite est importante et inversement. Pour un prix de l'actif sous-jacent fixé à 1260.36 et un prix d'exercice proche de cette valeur, la volatilité est assez faible. En termes financiers, pour les valeurs de K comprises entre 1000 et 1300, nous sommes dans un marché dit à la monnaie.

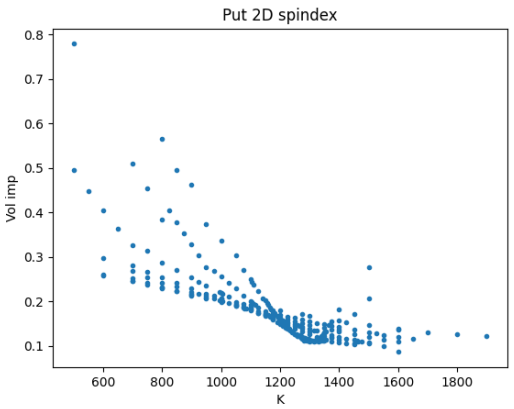
Put en 2D

Code :

```
#Graphique du Put 2D
def Put_2D():
    eps = 0.0001  # seuil pour la vérification
    t = 0
    q = 0.0217
    S0 = 1260.36

    chemin = "/home/cytech/Desktop/Documents/ING3/Calibration/TP1/sp-index.txt"
    fichier = np.loadtxt(chemin,skiprows=1)
    N = len(fichier) # Calcul du nb de lignes pour initialiser tous les vecteurs de données

    #Initialisation des données
    S_0 = np.zeros(N)
    vol_imp = np.zeros(N)
    K = np.zeros(N)
    r = np.zeros(N)
    T = np.zeros(N)
    Pb = np.zeros(N)
    Pa = np.zeros(N)
    Put = np.zeros(N)

    #Chargement des données
    T = fichier[:,0]
    K = fichier[:,1]
    Pb = fichier[:,4]
    Pa = fichier[:,5]
    r = fichier[:,6]/100

    #Calcul du cours
    Put = (Pb + Pa)/2

    for i in range(N):
        #Actualisation du prix
        S_0[i] = S0*np.exp(-q * (T[i] - t))
        if verification2(Put[i],S_0[i],K[i],T[i], t, r[i]):
            sigma = []
            #Initialisation du sigma
            sigma.append(np.sqrt(2*abs((np.log(S_0[i]/K[i]) + r[i]*(T[i] - t)) / (T[i] - t))))
            #Algorithme de Newton
            while abs(F_Put(S_0[i], K[i], r[i], sigma[-1], T[i], t, Put[i])) > eps:
                sigma.append(sigma[-1] - F_Put(S_0[i], K[i], r[i], sigma[-1], T[i], t, Put[i]) / Vega(S_0[i], K[i], T[i], t, r[i], sigma[-1]))
                vol_imp[i] = sigma[-1]

        else :
            vol_imp[i] = 0

    # Suppression de tout les strikes et sigma ou la volatilité implicite est nullle
    volat_nules=np.where(vol_imp==0)
    K = np.delete(K,volat_nules)
    vol_imp = np.delete(vol_imp,volat_nules)

    plt.plot(K,vol_imp,".")
    plt.title("Put 2D spindex")
    plt.ylabel("Vol imp")
    plt.xlabel("K")
    plt.show()
```

Explications : Raisonnement analogue au Put en 3D.

Graphique affiché :



Interprétation :

Mêmes observations que pour le Call.

Question 4

```python
#Travail sur SP-INDEX
def data_extract():
    eps = 0.0001
    t = 0
    g = 0.0217
    S0 = 1260.36

    chemin = "/home/cytech/Desktop/Documents/ING3/Calibration/TP1/sp-index.txt"
    fichier = np.loadtxt(chemin,skiprows=1)
    N = len(fichier)

    S_0 = np.zeros(N)
    sigma_imp = np.zeros(N)

    T = fichier[:,0]
    K = fichier[:,1]
    Cb = fichier[:,2]
    Ca = fichier[:,3]
    Pb = fichier[:,4]
    Pa = fichier[:,5]
    r = fichier[:,6]/100

    Call = (Cb + Ca)/2
    Put = (Pb + Pa)/2

    Sj = 8*[0]
    ln_Sj = 8*[0]
    Nj = []
    Tj = []

    # Question 1

    # set() supprime tout les doublons dans la liste et list() les tries et les ordonnes donc il reste que les 8 échéance dans le code.
    Tj = list(set(T))
    res = []

    # Estimation de S par la parité Call - Put
    S = 0
    compteur_test = 0
    Test_t = np.array(T)
    for j in range(len(Tj)):
        for i in range(N):
            if (Tj[j] == Test_t[i]):
                S = S + (Call[i] - Put[i] + np.exp(-r[i]*Test_t[i])*K[i])
                compteur_test += 1
        res.append(S/compteur_test)
        S = 0
        compteur_test = 0

    # Question 2
    # On calcul le log de notre estimation précédente
    log = np.log(res)

    plt.plot(Tj,log)
    plt.xlabel("Maturité")
    plt.ylabel("Avec Dividende")
    plt.title("Logarithme de S")
    plt.show()
```

Code :

```python
# Question 3 et 4

# On fait une régression linéaire sur : ln(S) = β1*T+ β2
# pour vérifier nos calculs et on plot le résultat
T_q4 = np.array(Tj).reshape(-1, 1)
ln_S = np.array(log)

model = LinearRegression()
model.fit(T_q4, ln_S)

# β1 = -q
# β2 = ln(S0)
b1 = model.coef_[0]
b2 = model.intercept_

print(f"β1: {b1}, β2: {b2}")

ln_S_pred = model.predict(T_q4)

plt.figure()
plt.scatter(Tj, ln_S, label='Données Observées', color='blue')
plt.plot(Tj, ln_S_pred, color='red', label='Régression Linéaire')
plt.xlabel("Maturité T")
plt.ylabel("ln(S chapeau)")
plt.title("Régression Linéaire de ln(S chapeau) en fonction des maturités T")
plt.legend()
plt.show()
#Question6
s0_6 = np.exp(b2)
q_6 = -b1
print("Valeur théorique de S0 = 1260.36 et valeur calculée :", s0_6)
print("Valeur théorique de q = 0.0217 et valeur calculée :", q_6)

#Solutions analytiques
somme1 = sum(Tj[i] * log[i] for i in range(8))
somme2 = sum(log)
somme3 = sum(Tj)
somme4 = sum(Tj[i]**2 for i in range(8))
beta1_f = (somme1 - (somme2 * somme3)/8) / (somme4 - (somme3**2) / 8)
beta2_f = (somme2 * somme4 - (somme1*somme3)) / (8*somme4 - somme3**2)
print("Valeur de beta1 avec la formule :", beta1_f)
print("Valeur du dividende sachant que q = -beta1 :", -beta1_f)
print("Valeur de beta2 avec la formule :", beta2_f)
print("Valeur de S0 = exp(beta2) avec la formule :", np.exp(beta2_f))
```

<u>Explications :</u>

- La question 1 du code permet d'estimer Sj pour chaque maturité de 1 à 8. Tj permet de supprimer les doublons des maturités T. Nous calculons Sj à l'aide de cette formule :

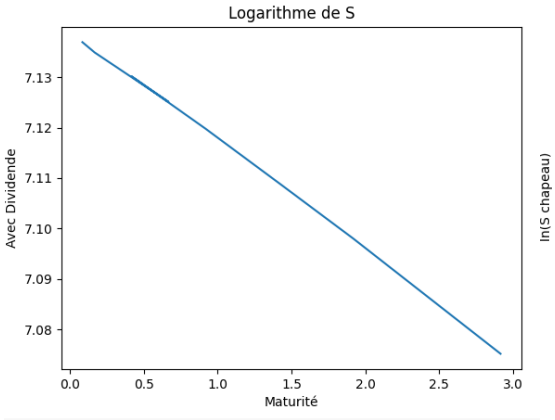
$$\hat{S}_j = \sum_{i=1}^{N_j} (Call(T_j, K_i) - Put(T_j, K_i) + e^{-rT_j} K_i)/N_j$$

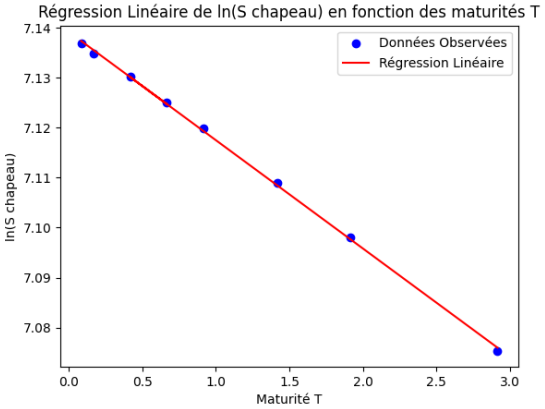
- La question 2 permet de tracer le logarithme de S_j en fonction de T_j à l'aide de la relation de parité :

$$ln(\hat{S}_j) = lnS_0 - qT_j$$

- Les questions 3 et 4 permettent d'appliquer une régression linéaire afin d'estimer Beta1 et Beta2. La régression linéaire permet de …
- La question 6 permet de trouver les valeurs de beta1 et beta2 grâce à la régression linéaire
- La partie suivante permet de calculer beta1 et beta2 à l'aide des formules du cours

Graphiques et résultats affichés :






```
β1: -0.02166466966128491, β2: 7.139157972457461
Valeur théorique de S0 = 1260.36 et valeur calculée : 1260.3666787091668
Valeur théorique de q = 0.0217 et valeur calculée : 0.02166466966128491
Valeur de beta1 avec la formule : -0.0216646696612862669661286 26
Valeur du dividende sachant que q = -beta1 : 0.02166466966128626
Valeur de beta2 avec la formule : 7.139157972457464667870917 02
Valeur de S0 = exp(beta2) avec la formule : 1260.3666787091702
```

Interprétations :

- Logarithme de S en fonction de T : nous constatons qu'avec un dividende, notre smile a le comportement d'une droite. De ce fait, nous pouvons effectuer une régression linéaire
- Les coefficients Beta1 et Beta2 ont une importance : le Beta1 correspond au coefficient directeur tandis que Beta2 correspond à l'ordonnée à l'origine. De plus, ces valeurs sont également utilisées pour retrouver les valeurs de q (dividende) et de S_0. En effet, $q = -beta1$ et $S_0 = e^{beta2}$. Ainsi nous trouvons, que ce soit par méthode analytique ou géométrique, les mêmes valeurs pour q et S_0 : notre modèle est donc correct.

III - Base de données “Google Origin”

Question 5

Code :

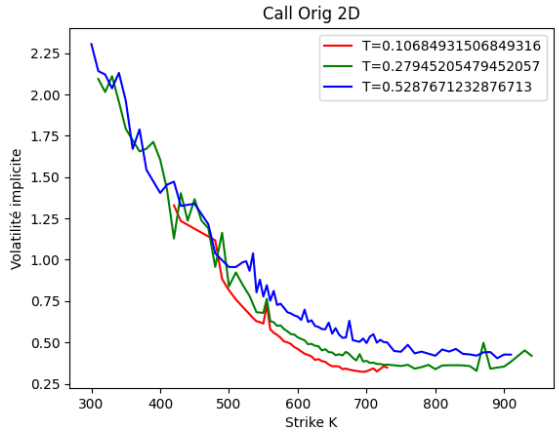
```
# Graphique du fichier GoogleOrig.xlsx
# smile de vol
def Google():
    eps = 0.0001
    t = 0
    q = 0.5
    S0 = 591.66

    chemin = "/home/cytech/Desktop/Documents/ING3/Calibration/TP1/GoogleOrig.xlsx"
    fichier = pd.read_excel(chemin,sheet_name='Orig', skiprows=1)
    N = len(fichier)

    # Initialisation des données
    S_0 = np.zeros(N)
    vol_imp = np.zeros(N)
    K = np.zeros(N)
    r = np.zeros(N)
    T = np.zeros(N)
    Call = np.zeros(N)

    #Chargement des données
    T = fichier['Time'].values
    K = fichier['Strike'].values
    Call = fichier['Call Price'].values

    for i in range(N):
        # Actualisation du prix
        S_0[i] = S0 * np.exp(-q * T[i])
        if verification(Call[i],S_0[i],K[i],T[i], t, r[i]):
            #Initialisation du sigma
            sigma = [np.sqrt(2 * abs((np.log(S_0[i] / K[i]) + r[i] * T[i]) / T[i]))]
            # Algorithme de Newton
            while abs(F(S_0[i], K[i], r[i], sigma[-1], T[i], t, Call[i])) > eps:
                sigma.append(sigma[-1] - F(S_0[i],K[i],r[i],sigma[-1],T[i],t,Call[i])/Vega(S_0[i],K[i], T[i], t, r[i],sigma[-1]))
                vol_imp[i] = sigma[-1]
        else:
            vol_imp[i] = 0
```

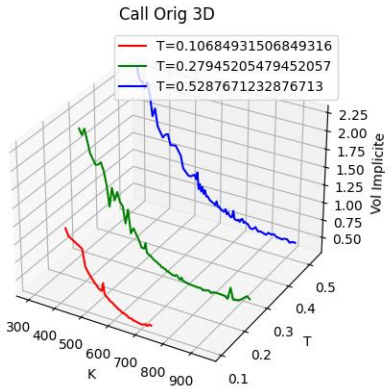
```
    # Suppression de tout les strikes et sigma ou la volatilité implicite est nullle
    volat_nules = np.where(vol_imp == 0)
    K = np.delete(K, volat_nules)
    T = np.delete(T, volat_nules)
    vol_imp = np.delete(vol_imp, volat_nules)

    # Tracé 2D avec couleur par maturité
    val_T = np.unique(T)
    colors = ['r', 'g', 'b']
    for i, t_val in enumerate(val_T):
        j = np.where(T == t_val)
        plt.plot(K[j], vol_imp[j], color=colors[i % len(colors)], label=f"T={t_val}")

    plt.title("Call Orig 2D")
    plt.ylabel("Volatilité implicite")
    plt.xlabel("Strike K")
    plt.legend()
    plt.show()

    # Tracé 3D
    fig = plt.figure()
    ax = fig.add_subplot(111, projection='3d')
    for i, t_val in enumerate(val_T):
        j = np.where(T == t_val)
        ax.plot(K[j], T[j], vol_imp[j], color=colors[i % len(colors)], label=f"T={t_val}")

    ax.set_title("Call Orig 3D")
    ax.set_xlabel("K")
    ax.set_ylabel("T")
    ax.set_zlabel("Vol Implicite")
    ax.legend()
    plt.show()
```

Explications :

Raisonnement analogue au précédent code concernant les Call. Seule l'extraction des données est différente car le format du fichier est différent.

Graphiques affichés :



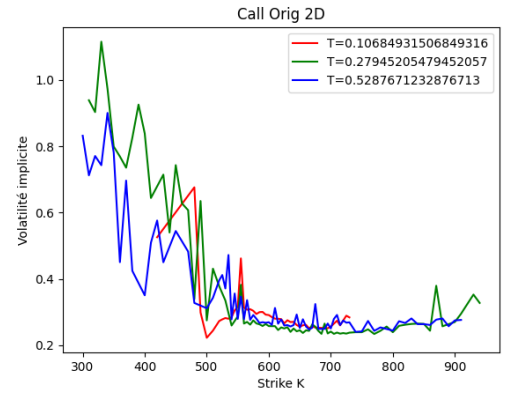


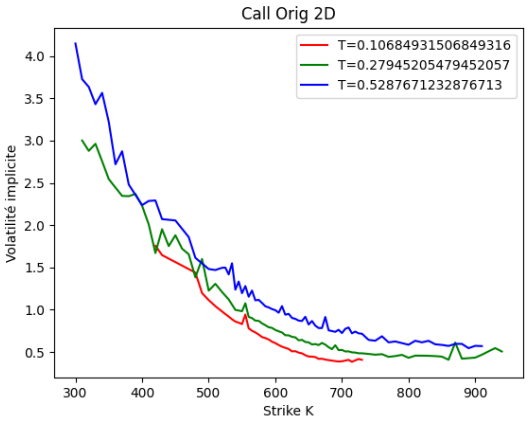
Interprétation :

Pour une valeur de q fixée à 0.5, nous avons de jolis smiles. En deux dimensions, nous pouvons remarquer une bonne superposition des courbes. Nous avons le même cas pour le graphique en trois dimensions.

Qu'en est-il pour d'autres valeurs de q ?

Prenons deux valeurs de q extrêmes, à savoir q=0 et q=0.99. En deux dimensions, nous obtenons :





Nous constatons une large différence sur l'axe des ordonnées qui prend en compte la volatilité implicite. En effet, pour un taux de dividende égal à 0.99, nous avons des valeurs pouvant être quatre fois plus élevées. Ainsi, les dividendes ont un important impact sur la

valeur des options. Lorsque q est élevé, nous nous attendons à une valeur du sous-jacent moins élevée. De ce fait, la volatilité implicite est plus élevée afin de compenser le risque.

IV - Base de données “Spx-quotedata”

Question 6-7-8

Code :

```
def Quote_Call():
    eps = 0.0001
    t = 0
    q = 0.0
    S0 = 3932.69

    chemin = "/home/cytech/Desktop/Documents/ING3/Calibration/TP1/spx_quotedata.csv"
    fichier = pd.read_csv(chemin,header=0,sep=',')
    N = len(fichier)

    # Initialisation des données
    S_0 = np.zeros(N)
    vol_imp = np.zeros(N)
    K = np.zeros(N)
    r = np.zeros(N)
    T = np.zeros(N)
    Cb = np.zeros(N)
    Ca = np.zeros(N)
    Call = np.zeros(N)
    Pb = np.zeros(N)
    Pa = np.zeros(N)
    Put = np.zeros(N)
    format = '%d-%m-%y'
    Maturite = []

    date_ref = datetime.strptime(fichier['Expiration Date'].iloc[0], format)

    for i in range(N):
        # Chargement des données
        date = fichier['Expiration Date'].iloc[i]
        Maturite.append(datetime.strptime(date,format))
        diff = (Maturite[i] - date_ref).days
        T[i] = diff/365.25

        Cb[i] = fichier['Bid'].iloc[i]
        Ca[i] = fichier['Ask'].iloc[i]
        Pb[i] = fichier['Bid.1'].iloc[i]
        Pa[i] = fichier['Ask.1'].iloc[i]
        K[i] = fichier['Strike'].iloc[i]
        r[i] = 0.0255
        S_0[i] = S0

        #Calculs du cours
        Call[i] = (Cb[i] + Ca[i])/2
        Put[i] = (Pb[i] + Pa[i])/2
```

```python
    #Calculs du cours
    Call[i] = (Cb[i] + Ca[i])/2
    Put[i] = (Pb[i] + Pa[i])/2

    if verification(Call[i],S_0[i],K[i],T[i], t, r[i]):
        #Initialisation du sigma
        sigma = [np.sqrt(2 * abs((np.log(S_0[i] / K[i]) + r[i] * T[i]) / T[i]))]
        # Algorithme de Newton
        while abs(F(S_0[i], K[i], r[i], sigma[-1], T[i], t, Call[i])) > eps:
            sigma.append(sigma[-1] - F(S_0[i],K[i],r[i],sigma[-1],T[i],t,Call[i])/Vega(S_0[i],K[i], T[i], t, r[i],sigma[-1]))
            vol_imp[i] = sigma[-1]
    else:
        vol_imp[i] = 0


# Suppression de tout les données ou la volatilité implicite est nullle
volat_nules=np.where(vol_imp==0)
K = np.delete(K,volat_nules)
vol_imp = np.delete(vol_imp,volat_nules)
T = np.delete(T,volat_nules)
Call = np.delete(Call, volat_nules)
Put = np.delete(Put, volat_nules)
r = np.delete(r,volat_nules)

vol_p = []
K_p = []
vol_d = []
K_d = []

for i in range(12):
    vol_p.append(vol_imp[i])
    K_p.append(K[i])

plt.plot(K_p,vol_p,".")
plt.title("Quote Call 2D - 12 premiers")
plt.ylabel("Vol imp")
plt.xlabel("K")
plt.show()

# Découpage de sous ensemble dans une liste
Tj = list(set(T))
res = []
Test_t = np.array(T)
```

```python
# Estimation de S par la parité Call - Put
for j in range(len(Tj)):
    s_c = 0
    compteur_test = 0
    for i in range(len(Call)):
        if Tj[j] == Test_t[i]:
            s_c += (Call[i] - Put[i] + np.exp(-r[i] * (Test_t[i] )) * K[i])
            compteur_test += 1
    if compteur_test > 0:
        res.append(s_c / compteur_test)
    else:
        res.append(0)

log = np.log(res)

plt.plot(Tj,log,".")
plt.xlabel("Maturité")
plt.ylabel("Avec Dividende")
plt.title("Logarithme de S")
plt.show()

# On fait une régression linéaire sur : ln(S) = β1*T+ β2
# pour vérifier nos calculs et on plot le résultat
T_q4 = np.array(Tj).reshape(-1, 1)
ln_S = np.array(log)

model = LinearRegression()
model.fit(T_q4, ln_S)

# β1 = -q
# β2 = ln(S0)
beta1 = model.coef_[0]
beta2 = model.intercept_

print(f"β1: {beta1}, β2: {beta2}")

ln_S_pred = model.predict(T_q4)

plt.figure()
plt.scatter(Tj, ln_S, label='Données Observées', color='blue')
plt.plot(Tj, ln_S_pred, color='red', label='Régression Linéaire')
plt.xlabel("Maturité T")
plt.ylabel("ln(S chapeau)")
plt.title("Régression Linéaire de ln(S chapeau) en fonction des maturités T")
plt.legend()
plt.show()
```

```python
# Les 12 derniers smiles
for i in range(-12, 0):
    vol_d.append(vol_imp[i])
    K_d.append(K[i])


plt.plot(K_d,vol_d,".")
plt.title("Quote Call 2D - 12 derniers")
plt.ylabel("Vol imp")
plt.xlabel("K")
plt.show()

# Tous les smiles
plt.plot(K,vol_imp,".")
plt.title("Quote Call 2D - Tous")
plt.ylabel("Vol imp")
plt.xlabel("K")
plt.show()


# Plot en 3 dimensions
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(K, T, vol_imp, c='b', marker='o')

ax.set_title("Quote Call 3D")
ax.set_xlabel("K")
ax.set_ylabel("T")
ax.set_zlabel("Vol imp")

plt.show()
```

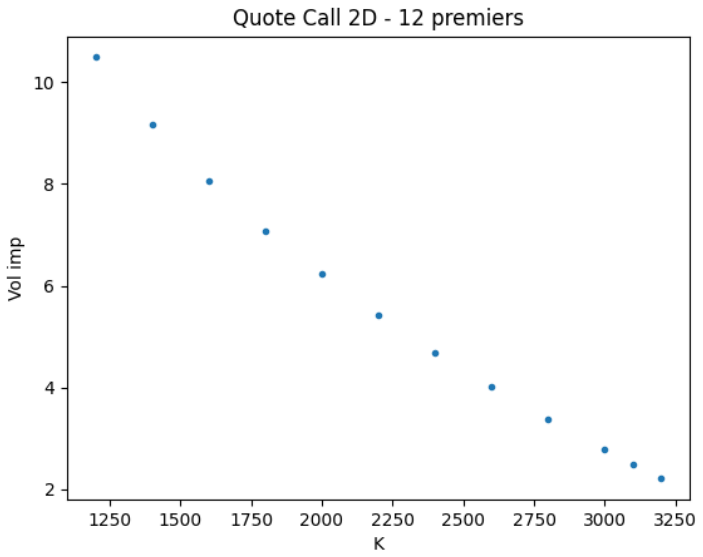
Question 6

<u>Explications :</u>

Raisonnement analogue aux autres codes portant sur les Call. Extraction des données différente. Pour les maturités : utilisation d'une variable format prenant le type de la date. Création d'une liste vide "Maturité" qui stockera les futures maturités. A l'aide de la fonction "strptime", création d'une variable prenant le rôle de la date référence. Au sein de la boucle, nous récupérons la date dans la colonne intitulée 'Expiration Date'. Nous ajoutons cette date convertie dans la liste "Maturité". Ensuite, nous calculons la différence entre chaque ligne et la date de référence. Finalement, nous divisons chaque maturité calculée par 365,22.

Pour afficher uniquement les 12 premiers smiles, nous ajoutons une boucle pour i allant de 1 à 12 puis utilisons de nouvelles listes vides.

Graphique affiché :



Résultat :

```
β1: 0.011838857922026028, β2: 8.274247168708529
```

Interprétation :

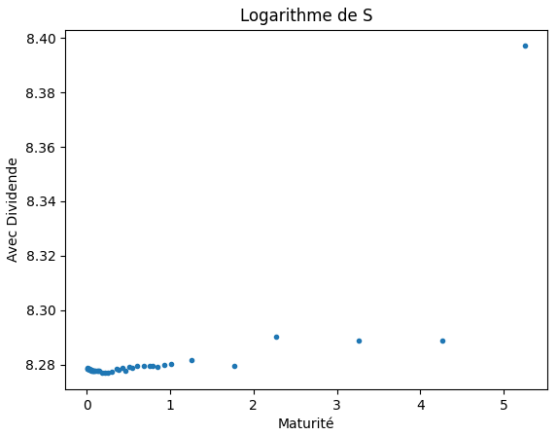
Nous constatons que les 12 premiers smiles pour un taux de dividende égal à zéro forment une droite. Le calcul par régression linéaire du taux de dividende sachant que $q = -beta1$ corrobore nos résultats. Et sachant que S_0 = exp(Beta2), nous pouvons tester notre Beta2 : $S0 = e^{8.274247229893307} = 3921,57$. Pour rappel notre valeur de S0 vaut 3932,69.

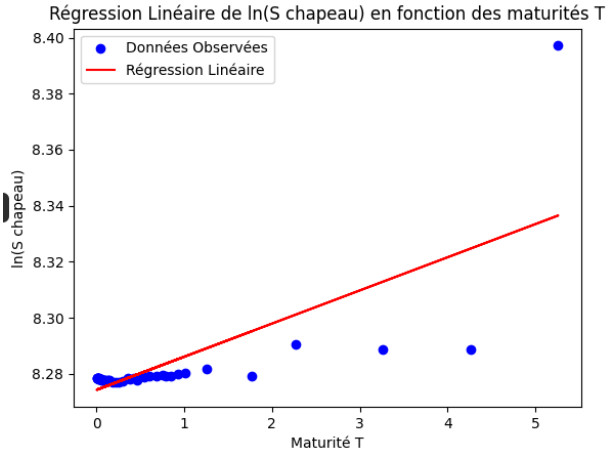
Question 7

Explications :

Nous devons donc calculer à nouveau S_j et son logarithme, nous reprenons donc le code utilisé auparavant en veillant à vérifier que les variables ont la même dimension.

Graphiques affichés :





Interprétation :

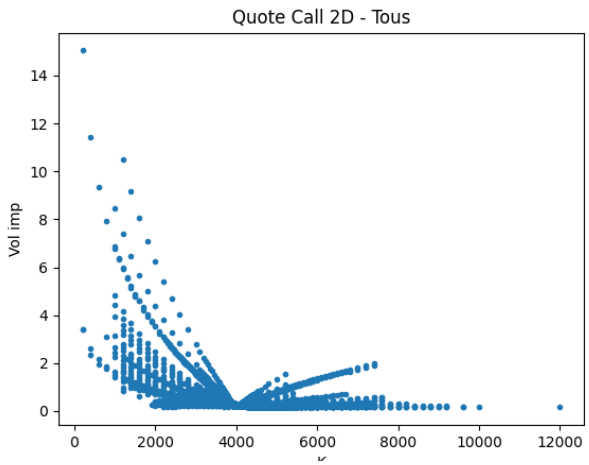
La régression linéaire nous indique une estimation pour Beta1 et Beta2, essentiels pour retrouver les valeurs du taux de dividende q et de S0.

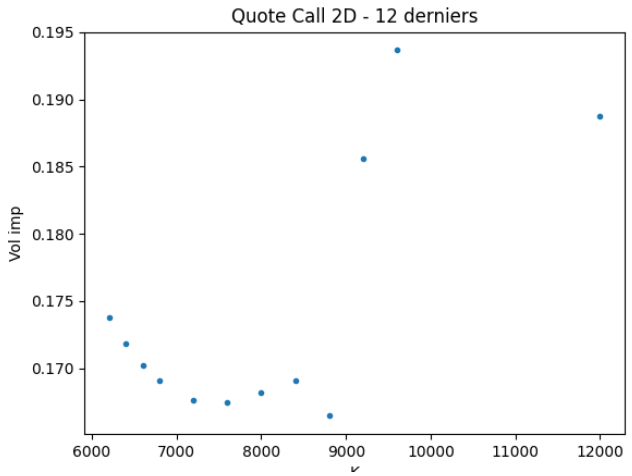
Question 8

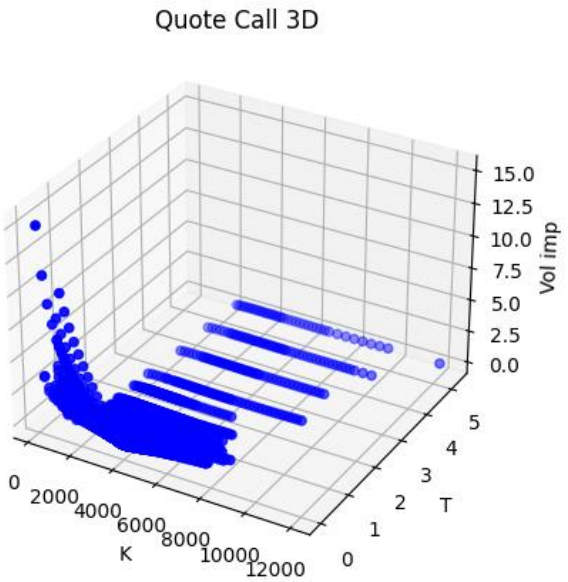
Explications :

Même raisonnement que précédemment. Nous codons normalement pour afficher l'ensemble des smiles. Pour les 12 derniers, nous utilisons une boucle analogue à celle des 12 premiers.

Graphiques affichés :







Question 9

<u>Code :</u>

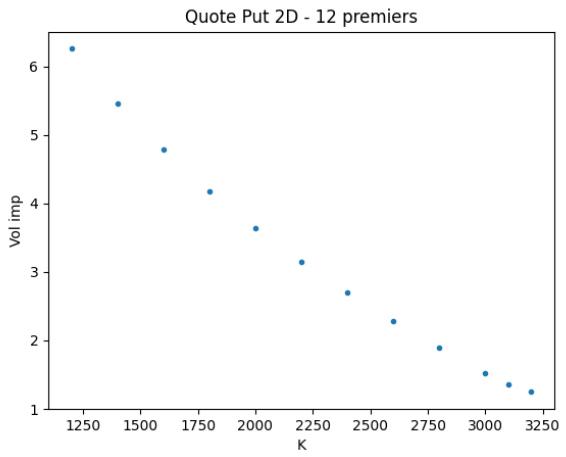
```
def Quote_Put():
    eps = 0.0001
    t = 0
    q = 0.0
    S0 = 3932.69

    chemin = "/home/cytech/Desktop/Documents/ING3/Calibration/TP1/spx_quotedata.csv"
    fichier = pd.read_csv(chemin,header=0,sep=',')
    N = len(fichier)

    # Initialisation des données
    S_0 = np.zeros(N)
    vol_imp = np.zeros(N)
    K = np.zeros(N)
    r = np.zeros(N)
    T = np.zeros(N)
    Cb = np.zeros(N)
    Ca = np.zeros(N)
    Call = np.zeros(N)
    Pb = np.zeros(N)
    Pa = np.zeros(N)
    Put = np.zeros(N)
    format = '%d-%m-%y'
    Maturite = []

    date_ref = datetime.strptime(fichier['Expiration Date'].iloc[0], format)

    for i in range(N):
        # Chargement des données
        date = fichier['Expiration Date'].iloc[i]
        Maturite.append(datetime.strptime(date,format))
        diff = (Maturite[i] - date_ref).days
        T[i] = diff/365.25

        Cb[i] = fichier['Bid'].iloc[i]
        Ca[i] = fichier['Ask'].iloc[i]
        Pb[i] = fichier['Bid.1'].iloc[i]
        Pa[i] = fichier['Ask.1'].iloc[i]
        K[i] = fichier['Strike'].iloc[i]
        r[i] = 0.0255
        S_0[i] = S0

        # Calcul du cours
        Put[i] = (Pb[i] + Pa[i])/2

        if verification2(Put[i],S_0[i],K[i],T[i], t, r[i]):
            #Initialisation du sigma
            sigma = [np.sqrt(2 * abs((np.log(S_0[i] / K[i]) + r[i] * T[i]) / T[i]))]
            # Algorithme de Newton
            while abs(F_Put(S_0[i], K[i], r[i], sigma[-1], T[i], t, Put[i])) > eps:
                sigma.append(sigma[-1] - F_Put(S_0[i], K[i], r[i], sigma[-1], T[i], t, Put[i]) / Vega(S_0[i], K[i], T[i], t, r[i], sigma[-1]))
            vol_imp[i] = sigma[-1]
        else:
            vol_imp[i] = 0

    # Suppression de tout les données ou la volatilité implicite est nullle
    volat_nules=np.where(vol_imp==0)
    K = np.delete(K,volat_nules)
    vol_imp = np.delete(vol_imp,volat_nules)
    T = np.delete(T,volat_nules)
```

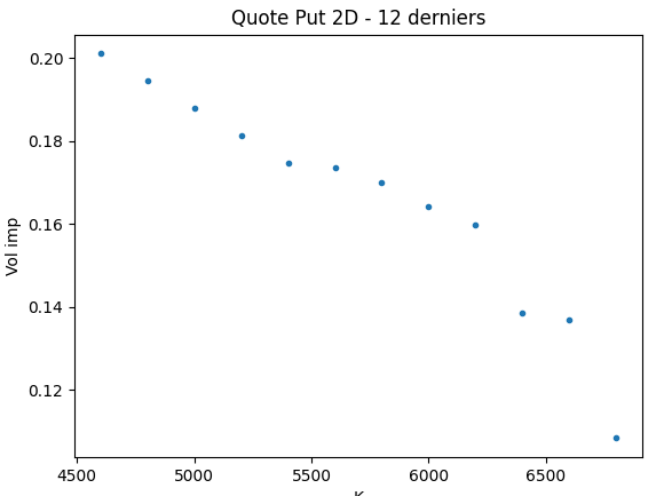
```python
vol_p = []
K_p = []
vol_d = []
K_d = []

for i in range(12):
    vol_p.append(vol_imp[i])
    K_p.append(K[i])

plt.plot(K_p,vol_p,".")
plt.title("Quote Put 2D - 12 premiers")
plt.ylabel("Vol imp")
plt.xlabel("K")
plt.show()

# Les 12 derniers smiles
for i in range(-12, 0):
    vol_d.append(vol_imp[i])
    K_d.append(K[i])

plt.plot(K_d,vol_d,".")
plt.title("Quote Put 2D - 12 derniers")
plt.ylabel("Vol imp")
plt.xlabel("K")
plt.show()

# Plot de tous les smiles
plt.plot(K,vol_imp,".")
plt.title("Quote Put 2D - Tous")
plt.ylabel("Vol imp")
plt.xlabel("K")
plt.show()

# Plot en 3 dimensions
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(K, T, vol_imp, c='b', marker='o')

ax.set_title("Quote Put 3D")
ax.set_xlabel("K")
ax.set_ylabel("T")
ax.set_zlabel("Vol imp")

plt.show()
```

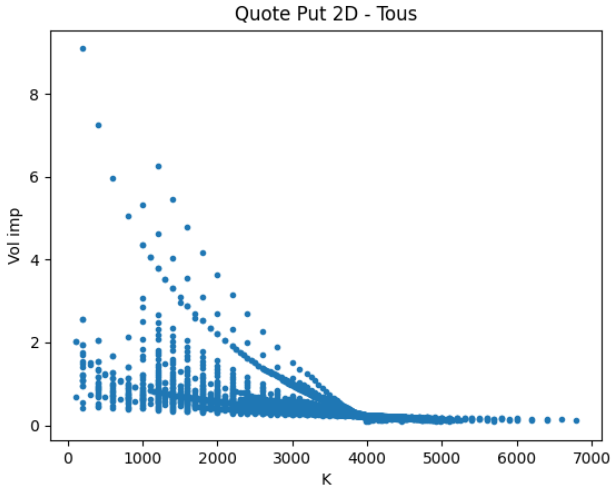
Explications :

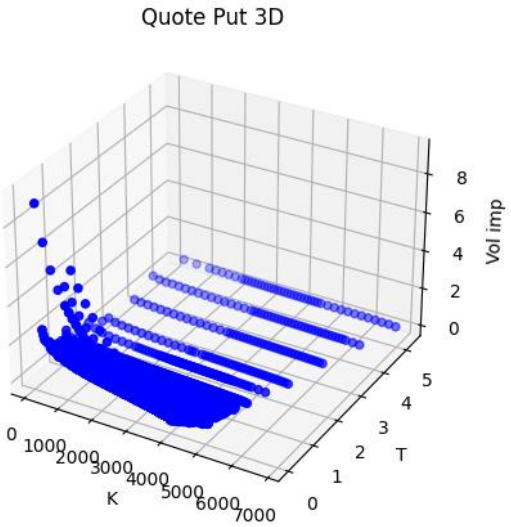
Même raisonnement que pour les codes liés au Put. Nous utilisons à nouveau le calcul sur les maturités.

Graphiques affichés :









V- Base de données "new-Data.csv"

Question 10-11

Code :

```
def data_extract2():
    eps = 0.0001
    t = 0
    q = 0.0
    S0 = 3932.69
    chemin = "/home/cytech/Desktop/Documents/ING3/Calibration/TP1/NEW_spx_quotedata.csv"
    format = '%y-%m-%d'

    # Lecture du fichier
    fichier = pd.read_csv(chemin, header=0, sep=',')
    N = len(fichier)

    # Initialisation
    S_0 = np.full(N, S0)
    vol_imp = np.zeros(N)
    K = fichier['Strike'].values
    T = np.zeros(N)
    r = np.full(N, 0.05)
    Call = ((fichier['Bid'] + fichier['Ask']) / 2).values
    Put = ((fichier['Bid.1'] + fichier['Ask.1']) / 2).values
    Maturite = [datetime.strptime(d, format) for d in fichier['Expiration Date']]
    date_ref = Maturite[0]
```

```python
# Calcul des maturités et de la volatilité implicite
for i in range(N):
    diff = (Maturite[i] - date_ref).days
    T[i] = diff / 365.25
    if T[i] <= 0:
        vol_imp[i] = 0
        continue
    if verification(Call[i], S_0[i], K[i], T[i], t, r[i]):
        try:
            #Initialisation du sigma
            sigma = [np.sqrt(2 * abs((np.log(S_0[i] / K[i]) + r[i] * T[i]) / T[i]))]
            #Algorithme de Newton
            for _ in range(50):  # J'ai instauré une limite de sécurité sur les itérations
                f_val = F(S_0[i], K[i], r[i], sigma[-1], T[i], t, Call[i])
                vega_val = Vega(S_0[i], K[i], T[i], t, r[i], sigma[-1])
                if vega_val == 0 or np.isnan(vega_val) or np.isinf(vega_val):
                    break
                sigma_new = sigma[-1] - f_val / vega_val
                if np.isnan(sigma_new) or np.isinf(sigma_new):
                    break
                sigma.append(sigma_new)
                if abs(f_val) < eps:
                    break
            vol_imp[i] = sigma[-1]
        except Exception:
            vol_imp[i] = 0
    else:
        vol_imp[i] = 0

# Suppression des lignes avec volatilité nulle
valid = (vol_imp > 0) & (~np.isnan(vol_imp)) & (~np.isinf(vol_imp))
K = K[valid]
T = T[valid]
vol_imp = vol_imp[valid]
Call = Call[valid]
Put = Put[valid]
r = r[valid]

# Plot smile 2D
plt.figure()
plt.plot(K, vol_imp, ".")
plt.title("Smile de volatilité implicite")
plt.xlabel("Strike (K)")
plt.ylabel("Volatilité implicite")
plt.grid(True)
plt.show()
```

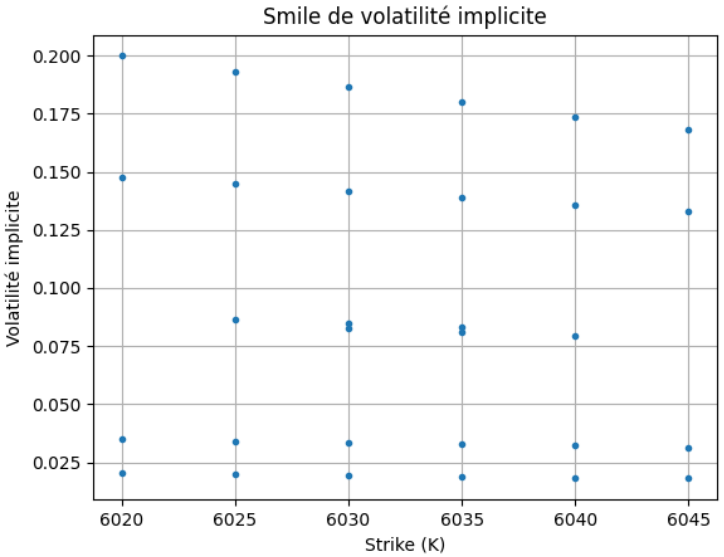
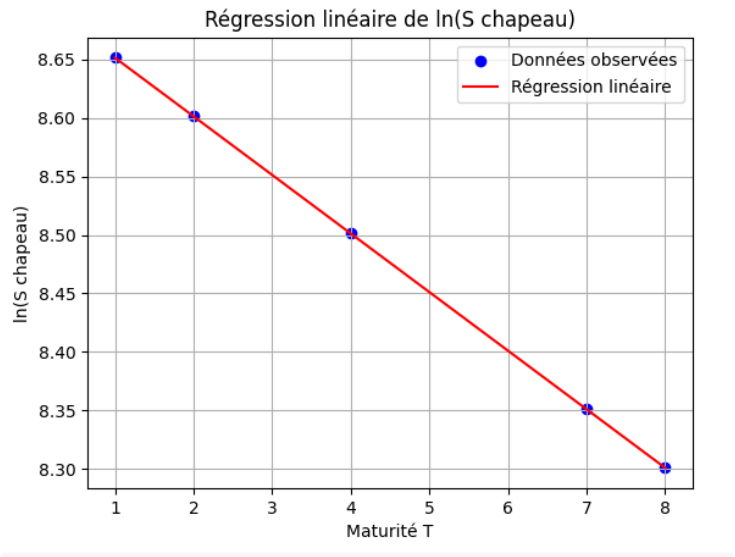
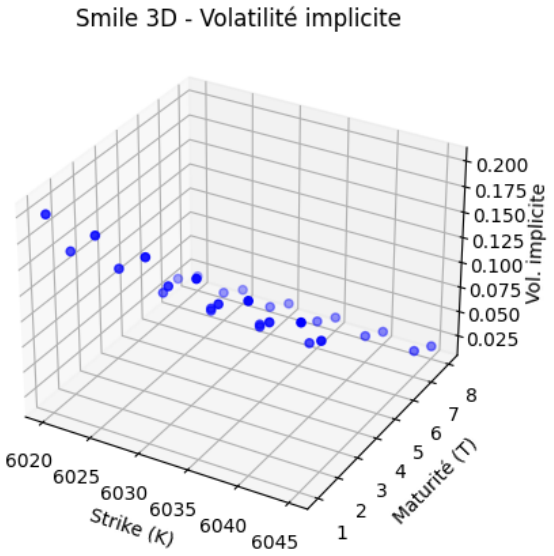
```python
# Découpage de sous ensemble dans une liste
T_unique = sorted(set(T))
S_hat = []

# Estimation de S par la parité Call - Put
for Tj in T_unique:
    indices = np.where(T == Tj)[0]
    if len(indices) > 0:
        s_vals = Call[indices] - Put[indices] + np.exp(-r[indices] * T[indices]) * K[indices]
        s_moy = np.mean(s_vals)
        if s_moy > 0:
            S_hat.append(s_moy)
        else:
            S_hat.append(np.nan)
    else:
        S_hat.append(np.nan)

S_hat = np.array(S_hat)
valid_ln = ~np.isnan(S_hat) & (S_hat > 0)
ln_S = np.log(S_hat[valid_ln])
T_arr = np.array(T_unique)[valid_ln].reshape(-1, 1)

# On fait une régression linéaire sur : ln(S) = β1*T+ β2
# pour vérifier nos calculs et on plot le résultat
model = LinearRegression()
model.fit(T_arr, ln_S)

# β1 = -q
# β2 = ln(S0)
beta1 = model.coef_[0]
beta2 = model.intercept_
print(f"β1 = {beta1:.6f}, β2 = {beta2:.6f}")

# Plot regression linéaire
plt.figure()
plt.scatter(T_arr, ln_S, color='blue', label='Données observées')
plt.plot(T_arr, model.predict(T_arr), color='red', label='Régression linéaire')
plt.xlabel("Maturité T")
plt.ylabel("ln(S chapeau)")
plt.title("Régression linéaire de ln(S chapeau)")
plt.legend()
plt.grid(True)
plt.show()

# Plot smile 3D
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(K, T, vol_imp, c='blue', marker='o')
ax.set_title("Smile 3D - Volatilité implicite")
ax.set_xlabel("Strike (K)")
ax.set_ylabel("Maturité (T)")
ax.set_zlabel("Vol. implicite")
plt.show()
```

<u>Explications :</u>

Analyse des smiles pour S0 = 3932,69 :

Nos différents tests montrent comment la volatilité implicite varie par rapport aux maturités et aux prix d'exercices. Les options avec un strike proche de S0 sont à la monnaie donc la volatilité implicite est plutôt basse. A contrario, plus les options avec des prix d'exercice éloignés du prix du sous-jacent auront une volatilité implicite élevée. Cette augmentation de la volatilité implicite indique une perception du risque plus élevée.

Les maturités sont également importantes et nos graphiques nous permettent de le constater. En effet, dans le cas de maturités courtes, les smiles sont très marqués lorsqu'ils sont aussi liés à des prix d'exercice très inférieurs au prix du sous-jacent. Pour des maturités plus longues, le marché absorbe les risques et diminue donc les variations des smiles et ce, peu importe l'éloignement du prix d'exercice à celui du sous-jacent.

Tracer les 12 premiers ou 12 derniers smiles des Call et Put nous permettent également de remarquer que les variations de la volatilité implicite ont tendance à se rapprocher d'une forme linéaire sur le long terme. Les Put ont également une volatilité implicite légèrement plus élevée que les Call lorsqu'on est hors de la monnaie.

Graphiques affichés :